

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на разработку MVP автоматизированной системы управления постоянно действующей научной конференцией

Проект: автоматизированная система приёма, проверки, оформления и публикации материалов постоянно действующей научной конференции по искусственному интеллекту

Версия: 1.0

Срок выполнения MVP: 1 месяц

Формат разработки: групповой проект студентов на практике

Целевая периодичность конференции: не менее 4 выпусков в год

Тип материалов: тезисы, короткие научные сообщения, материалы студенческих и магистерских исследований

Основной принцип: максимальная автоматизация рутинных операций с обязательным финальным решением человека

1. Назначение проекта

Разрабатываемая система предназначена для автоматизации работы постоянно действующей научной конференции. Система должна принимать материалы авторов, приводить их к единому шаблону конференции, выполнять несколько уровней автоматической проверки, формировать замечания автору, отправлять оформленный вариант на согласование, после чего передавать материал редактору для финального решения.

Система не должна самостоятельно принимать окончательное решение о публикации. Все автоматические проверки должны носить рекомендательный характер. Финальное решение принимает человек через редакторский интерфейс.

Главная цель MVP — создать воспроизводимый цифровой конвейер:

загрузка материала автором
↓
извлечение структуры из файла
↓
приведение к шаблону конференции
↓
автоматические проверки
↓
формирование отчёта замечаний
↓
отправка автору на согласование
↓
получение подтверждения или исправленной версии

↓
проверка редактором
↓
публикация в выпуске конференции
↓
сборка электронного сборника

2. Общая идея системы

Система должна работать как модульная платформа. В ней есть общее ядро, которое управляет заявками, файлами, статусами и этапами обработки. Отдельные модули подключаются к ядру как независимые обработчики.

Каждый материал проходит по цепочке этапов:

1. Приём заявки.
2. Загрузка файла.
3. Распознавание структуры документа.
4. Извлечение метаданных.
5. Приведение материала к шаблону конференции.
6. Автоматическая формальная проверка.
7. Автоматическая смысловая проверка.
8. Проверка на недопустимое содержание.
9. Проверка на закрытые или чувствительные сведения.
10. Формирование отчёта для автора.
11. Согласование автором.
12. Проверка редактором.
13. Включение в выпуск.
14. Генерация PDF отдельного материала.
15. Генерация электронного сборника.
16. Публикация в публичном архиве конференции.

Система должна быть спроектирована так, чтобы в будущем можно было добавлять новые этапы обработки без переписывания всего проекта. Например, можно будет добавить новый модуль проверки, новый этап согласования, новый внешний сервис, новый тип документа или новый формат выгрузки.

3. Что должно получиться в MVP

MVP должен демонстрировать полный путь одного материала от загрузки до включения в выпуск конференции.

Минимально система должна уметь:

1. Принимать DOCX-файл от автора.
 2. Принимать материал, оформленный не строго по шаблону.
 3. Извлекать из материала название, авторов, организацию, аннотацию, ключевые слова, основной текст, список литературы.
 4. Приводить материал к единому DOCX-шаблону конференции.
 5. Генерировать PDF-версию материала.
 6. Выполнять набор автоматических проверок.
 7. Формировать понятный отчёт замечаний.
 8. Показывать автору оформленную версию.
 9. Позволять автору подтвердить материал или загрузить исправленную версию.
 10. Показывать редактору карточку материала.
 11. Позволять редактору принять, отклонить или отправить материал на доработку.
 12. Формировать список принятых материалов текущего выпуска.
 13. Собирать электронный сборник выпуска.
 14. Хранить историю действий по материалу.
 15. Иметь README, тестовые файлы и демонстрационный сценарий.
-

4. Границы MVP

В MVP обязательно включается:

DOCX как основной входной формат;
ручной финальный контроль редактора;
единый шаблон конференции;
автоматическое оформление материала;
автоматическая проверка структуры;
автоматическая проверка текста нейросетевой моделью;
статусы заявки;
личный кабинет автора в упрощённом виде;
редакторская панель;
формирование PDF;
формирование сборника выпуска;
публичная страница архива выпусков.

В MVP не включается:

полноценный личный кабинет с юридически значимой авторизацией;
сложная система ролей;
поддержка старого формата .doc;
OCR сканированных PDF;
гарантированное исправление любых повреждённых Word-файлов;
полноценная антиплагиатная система;
автоматическое научное рецензирование;
автоматическое окончательное решение о публикации;

сложная интеграция с внешними библиотечными системами;
массовая почтовая рассылка промышленного уровня;
мобильное приложение.

5. Роли пользователей

В MVP достаточно предусмотреть три роли.

5.1. Автор

Автор загружает материал, смотрит статус, получает отчёт замечаний, скачивает оформленную версию и подтверждает согласие с итоговым вариантом.

Функции автора:

- создать заявку;
- заполнить сведения об авторах;
- загрузить DOCX-файл;
- посмотреть найденные системой метаданные;
- скачать оформленную версию;
- посмотреть отчёт проверок;
- загрузить исправленную версию;
- подтвердить итоговый вариант;
- видеть текущий статус материала.

5.2. Редактор

Редактор просматривает все заявки, видит результаты автоматических проверок, принимает финальное решение и управляет выпуском.

Функции редактора:

- видеть список материалов;
- фильтровать материалы по статусам;
- открывать карточку материала;
- скачивать исходный файл;
- скачивать оформленный DOCX;
- скачивать PDF;
- видеть отчёт проверок;
- видеть историю действий;
- отправлять материал на доработку;
- принимать материал;
- отклонять материал;
- включать материал в выпуск;
- запускать сборку выпуска.

5.3. Администратор системы

В MVP администратор может быть совмещён с редактором. Его задача — настраивать конференцию, выпуски, шаблоны и этапы обработки.

Функции администратора:

- создавать выпуск конференции;
- указывать даты приёма материалов;
- загружать шаблон конференции;
- включать и отключать этапы обработки;
- просматривать системные логи;
- перезапускать обработку материала;
- управлять публичным архивом выпусков.

6. Статусы материала

Каждый материал должен иметь понятный статус. Статусы нужны для автора, редактора и автоматического конвейера.

Рекомендуемый набор статусов:

- draft — черновик заявки;
- uploaded — файл загружен;
- structure_extracted — структура извлечена;
- formatted — материал приведён к шаблону;
- auto_checking — выполняются автоматические проверки;
- auto_checked — автоматические проверки завершены;
- needs_author_review — отправлено автору на согласование;
- author_confirmed — автор подтвердил итоговый вариант;
- needs_revision — требуется доработка;
- editor_review — находится на проверке редактора;
- accepted — принято;
- rejected — отклонено;
- included_in_issue — включено в выпуск;
- published — опубликовано в архиве;
- error — ошибка обработки.

История изменения статусов должна сохраняться.

Пример записи истории:

```
{
  "submission_id": "SUB-2026-Q1-00015",
  "from_status": "formatted",
  "to_status": "auto_checking",
  "changed_by": "system",
  "changed_at": "2026-06-29T12:30:00",
  "comment": "Запущены автоматические проверки материала."
}
```

7. Основной пользовательский сценарий

7.1. Сценарий автора

1. Автор открывает страницу конференции.
2. Нажимает «Подать материал».
3. Заполняет сведения:
 - название материала;
 - ФИО авторов;
 - организация;
 - e-mail;
 - научный руководитель, если есть;
 - секция конференции;
 - ключевые слова.
4. Загружает DOCX-файл.
5. Система создаёт заявку.
6. Система извлекает структуру материала.
7. Система показывает найденные данные.
8. Система приводит материал к шаблону конференции.
9. Система выполняет автоматические проверки.
10. Автор получает отчёт замечаний.
11. Автор скачивает оформленный DOCX/PDF.
12. Автор подтверждает итоговый вариант или загружает исправленную версию.
13. После подтверждения материал переходит редактору.

7.2. Сценарий редактора

1. Редактор открывает панель заявок.
2. Видит список материалов по текущему выпуску.
3. Открывает карточку конкретного материала.
4. Просматривает:
 - исходный файл;
 - оформленную версию;

- PDF;
 - метаданные;
 - результаты автоматических проверок;
 - историю действий.
5. Принимает одно из решений:
- принять;
 - отправить на доработку;
 - отклонить;
 - отложить решение.
6. После принятия материал попадает в список текущего выпуска.
7. Редактор запускает сборку выпуска.
8. Система формирует электронный сборник.
-

8. Общий контракт данных

Все модули системы должны обмениваться данными через единый JSON-контракт. Это нужно, чтобы разные студенты могли независимо разрабатывать свои части.

8.1. Submission

```
{
  "submission_id": "SUB-2026-Q1-00001",
  "conference_id": "ai_quarterly_conf",
  "issue_id": "2026_q1",
  "status": "uploaded",
  "created_at": "2026-06-29T12:00:00",
  "updated_at": "2026-06-29T12:10:00",
  "author_contact": {
    "full_name": "Иванов Иван Иванович",
    "email": "ivanov@example.com",
    "organization": "ТГТУ"
  },
  "metadata": {
    "title_ru": "Применение нейросетей для анализа учебных текстов",
    "title_en": "",
    "authors": [
      {
        "full_name": "Иванов И.И.",
        "organization": "ТГТУ",
        "email": "ivanov@example.com"
      }
    ]
  },
  "supervisor": "Петров П.П.",
  "section": "Искусственный интеллект в образовании",
}
```

```

    "keywords_ru": ["нейросети", "образование", "анализ текста"],
    "abstract_ru": ""
  },
  "files": {
    "original_docx": "storage/submissions/SUB-2026-Q1-00001/original.docx",
    "formatted_docx": "",
    "formatted_pdf": "",
    "check_report": ""
  },
  "checks": [],
  "editor_decision": null
}

```

8.2. Результат автоматической проверки

```

{
  "check_id": "semantic_quality_check",
  "title": "Смысловая проверка материала",
  "status": "completed",
  "risk_level": "medium",
  "score": 0.72,
  "summary": "Материал в целом соответствует тематике конференции, но содержит слабое описание методики.",
  "warnings": [
    {
      "code": "weak_methodology",
      "message": "Недостаточно подробно описана методика исследования.",
      "location": "Раздел: Основной текст",
      "recommendation": "Добавить описание данных, метода и критериев оценки."
    }
  ],
  "errors": [],
  "raw_model_response_path": "storage/submissions/SUB-2026-Q1-00001/checks/semantic_quality_check.json"
}

```

8.3. Итоговый отчёт проверок

```

{
  "submission_id": "SUB-2026-Q1-00001",
  "overall_status": "needs_attention",
  "overall_risk_level": "medium",
  "checks": [
    {
      "check_id": "formal_structure_check",
      "status": "passed",
      "risk_level": "low"
    },
    {
      "check_id": "semantic_quality_check",
      "status": "warning",
      "risk_level": "medium"
    }
  ],
}

```

```
{
  "check_id": "restricted_content_check",
  "status": "passed",
  "risk_level": "low"
},
"author_message": "Материал может быть направлен на согласование после
исправления замечаний по методике исследования.",
"editor_message": "Требуется ручная проверка раздела с описанием методики."
}
```

9. Архитектура системы

Для MVP рекомендуется использовать простую архитектуру:

```
conference_system/
  app/
    main.py
    config.py
    database.py

  core/
    models.py
    workflow.py
    status_machine.py
    file_storage.py
    event_log.py
    plugin_registry.py
    base_stage.py

  submissions/
    service.py
    repository.py
    schemas.py

  docx_processing/
    extractor.py
    formatter.py
    template_engine.py
    pdf_exporter.py
    metadata_parser.py

  checks/
    formal_structure/
      stage.py
      prompts.py
    semantic_quality/
      stage.py
      prompts.py
```

```
prohibited_content/  
  stage.py  
  prompts.py  
restricted_information/  
  stage.py  
  prompts.py  
formatting_quality/  
  stage.py  
  prompts.py  
  
author_ui/  
  pages.py  
  forms.py  
  
editor_ui/  
  dashboard.py  
  submission_card.py  
  issue_manager.py  
  
issue_builder/  
  issue_service.py  
  pdf_merger.py  
  toc_generator.py  
  archive.py  
  
notifications/  
  email_sender.py  
  templates.py  
  
samples/  
  templates/  
  input_materials/  
  expected_outputs/  
  
storage/  
  submissions/  
  issues/  
  logs/  
  
tests/  
  test_docx_extraction.py  
  test_template_formatting.py  
  test_checks.py  
  test_workflow.py  
  
README.md  
requirements.txt  
.env.example
```

Для быстрого MVP можно использовать:

Python 3.11+
FastAPI или Streamlit
SQLite
SQLAlchemy
Pydantic
python-docx
docxtempl
LibreOffice headless
pandas
jinja2
httpx
pytest

Если команда слабая, лучше начинать со Streamlit. Если команда сильнее, лучше делать FastAPI + простой веб-интерфейс.

10. Модульная система этапов обработки

Система должна поддерживать подключаемые этапы обработки. Каждый этап получает заявку, выполняет действие и возвращает результат.

Пример базового класса этапа:

```
from abc import ABC, abstractmethod
from typing import Any

class BaseWorkflowStage(ABC):
    stage_id: str
    title: str
    description: str
    enabled: bool = True

    @abstractmethod
    def run(self, submission: dict[str, Any]) -> dict[str, Any]:
        """
        Выполняет этап обработки материала.
        Возвращает структурированный результат.
        """
        pass
```

Пример регистрации этапов:

```
WORKFLOW_STAGES = [
    "extract_metadata",
    "format_to_template",
    "formal_structure_check",
    "semantic_quality_check",
    "prohibited_content_check",
    "restricted_information_check",
    "author_approval",
```

```
    "editor_review",  
    "include_in_issue"  
]
```

Каждый этап должен:

- иметь уникальный `stage_id`;
- иметь человекочитаемое название;
- принимать `submission_id`;
- читать нужные файлы из хранилища;
- сохранять результат в JSON;
- возвращать статус `success`, `warning`, `failed` или `skipped`;
- писать запись в журнал событий;
- не ломать всю систему при ошибке;
- показывать понятное сообщение пользователю.

11. Автоматические проверки

Система должна иметь несколько уровней проверки. Проверки не должны автоматически удалять или принимать материал. Они должны формировать отчёт для автора и редактора.

11.1. Формальная проверка

Проверяет наличие обязательных элементов:

- название;
- авторы;
- организация;
- e-mail контактного автора;
- аннотация;
- ключевые слова;
- основной текст;
- список литературы;
- объём материала;
- соответствие шаблону;
- отсутствие пустых разделов;
- корректность структуры заголовков.

Пример результата:

```
{  
  "check_id": "formal_structure_check",  
  "status": "warning",  
  "warnings": [  
    {  
      "code": "missing_keywords",  
      "message": "Не найдены ключевые слова.",  
      "recommendation": "Добавить 3-7 ключевых слов."  
    }  
  ],  
}
```

```
"errors": []  
}
```

11.2. Проверка тематического соответствия

Проверяет, относится ли материал к тематике конференции.

Система должна определить:

- основную тему материала;
- наличие связи с искусственным интеллектом;
- соответствие выбранной секции;
- наличие исследовательской или проектной составляющей;
- отсутствие полного несоответствия тематике конференции.

11.3. Смысловая проверка

Проверяет, не является ли материал бессвязным, случайно сгенерированным или бессодержательным.

Признаки риска:

- нет понятной цели работы;
- нет объекта исследования;
- нет метода;
- нет результата;
- текст состоит из общих фраз;
- абзацы слабо связаны между собой;
- название не соответствует содержанию;
- выводы не следуют из текста;
- есть противоречия внутри материала;
- много терминов без объяснения;
- материал похож на набор случайных утверждений.

11.4. Проверка на недопустимое содержание

Система должна выявлять материалы, которые требуют обязательной ручной проверки из-за потенциально недопустимого содержания.

Проверяются:

- призывы к незаконным действиям;
- агитационные и радикальные формулировки;
- инструкции по причинению вреда;
- опасные технические инструкции;
- оскорбительные или дискриминационные фрагменты;
- материалы, не относящиеся к научному содержанию;
- провокационные фрагменты, требующие ручной оценки.

При обнаружении риска система не должна публиковать материал автоматически. Материал должен получить статус, требующий ручной проверки редактором.

11.5. Проверка на закрытые и чувствительные сведения

Система должна выявлять признаки того, что в тексте могут быть сведения, которые нельзя публиковать без дополнительной проверки.

Проверяются:

- персональные данные третьих лиц;
- номера документов;
- адреса проживания;
- телефоны, не предназначенные для публикации;
- служебные внутренние сведения организации;
- описание неразрешённых внутренних процессов;
- упоминания закрытых объектов;
- чрезмерно подробные технические параметры чувствительных систем;
- непроверенные данные о людях или организациях.

Результат проверки должен быть осторожным. Система не утверждает, что материал точно содержит запрещённые сведения. Она только помечает фрагменты как требующие внимания редактора.

11.6. Проверка качества оформления

Проверяет:

- единый стиль заголовков;
- наличие подписей к рисункам;
- наличие подписей к таблицам;
- наличие ссылок на рисунки и таблицы в тексте;
- единообразие списка литературы;
- отсутствие слишком длинных абзацев;
- отсутствие лишних пустых строк;
- корректность переносов;
- корректность языка;
- наличие русской и английской версии названия, если требуется.

12. Пример промпта для смысловой проверки

Ты являешься помощником редактора студенческой научной конференции.

Тебе передан текст материала. Нужно проверить, является ли материал связным научным или учебно-исследовательским текстом.

Не принимай окончательное решение о публикации. Твоя задача — сформировать отчёт для редактора.

Проверь:

1. Есть ли цель работы.
2. Есть ли объект или предмет исследования.

3. Описан ли метод или подход.
4. Есть ли результат.
5. Соответствуют ли выводы основному тексту.
6. Нет ли внутренних противоречий.
7. Не состоит ли текст из общих фраз без содержания.
8. Соответствует ли название содержанию.
9. Соответствует ли текст тематике конференции по искусственному интеллекту.

Верни результат строго в JSON:

```
{
  "status": "passed | warning | failed",
  "risk_level": "low | medium | high",
  "score": 0.0,
  "summary": "...",
  "warnings": [
    {
      "code": "...",
      "message": "...",
      "location": "...",
      "recommendation": "..."
    }
  ],
  "editor_comment": "...",
  "author_comment": "..."
}
```

Текст материала:
{{ material_text }}

13. Пример промпта для проверки чувствительных сведений

Ты являешься помощником редактора научной конференции.

Тебе передан текст материала. Нужно найти фрагменты, которые могут требовать ручной проверки перед публикацией.

Проверяй только признаки риска. Не делай окончательных юридических выводов.

Нужно обратить внимание на:

1. Персональные данные третьих лиц.
2. Номера документов.
3. Адреса, телефоны и личные контакты.
4. Внутренние служебные сведения организаций.
5. Подробные описания закрытых или чувствительных объектов.
6. Непроверенные утверждения о людях и организациях.
7. Сведения, которые выглядят не предназначенными для открытой публикации.

Верни результат строго в JSON:

```
{
  "status": "passed | warning | failed",
  "risk_level": "low | medium | high",
  "summary": "...",
  "flagged_fragments": [
    {
      "fragment": "...",
      "risk_type": "...",
      "reason": "...",
      "recommendation": "..."
    }
  ],
  "editor_comment": "...",
  "author_comment": "..."
}
```

Текст материала:
{ { material_text } }

14. DOCX-обработка и шаблон конференции

Система должна уметь принимать материал в произвольном DOCX-оформлении и приводить его к шаблону конференции.

14.1. Что нужно извлечь из DOCX

название материала;
ФИО авторов;
организации авторов;
e-mail контактного автора;
сведения о научном руководителе;
аннотация;
ключевые слова;
основной текст;
разделы;
таблицы;
рисунки;
подписи к таблицам;
подписи к рисункам;
формулы, если есть;
список литературы.

14.2. Что нужно сформировать

formatted_material.docx;
formatted_material.pdf;

extracted_metadata.json;
formatting_report.json;
check_report.json;
author_report.docx или author_report.pdf.

14.3. Ограничения DOCX-модуля MVP

В MVP допускается:

- обрабатывать только .docx;
- не поддерживать старый .doc;
- не поддерживать сканированные PDF;
- не гарантировать идеальное сохранение сложной верстки;
- не обрабатывать сложные Word-формулы во всех случаях;
- не переносить комментарии и исправления Word;
- при ошибке извлечения показывать понятное предупреждение.

15. Шаблон конференционного материала

Для MVP нужно создать единый DOCX-шаблон конференции.

Рекомендуемая структура материала:

УДК, если есть

НАЗВАНИЕ МАТЕРИАЛА НА РУССКОМ ЯЗЫКЕ

Иванов И.И., Петров П.П.

Организация

Научный руководитель: Сидоров С.С.

Аннотация.

Текст аннотации.

Ключевые слова:

искусственный интеллект, машинное обучение, анализ данных

Основной текст материала.

Список литературы.

1. ...

2. ...

Дополнительно можно предусмотреть английские поля:

Title in English
Abstract in English
Keywords in English

Но для MVP это можно сделать необязательным.

16. Публичный архив конференции

Система должна иметь простую публичную страницу, где отображаются выпуски конференции.

Для каждого выпуска показывается:

название конференции;
год;
квартал;
название выпуска;
дата публикации;
список материалов;
авторы;
секция;
ссылка на PDF материала;
ссылка на PDF сборника.

Пример структуры архива:

```
/archive
  /2026
    /q1
      index.html
      collection.pdf
      materials/
        SUB-2026-Q1-00001.pdf
        SUB-2026-Q1-00002.pdf
```

17. Разбиение проекта между студентами

Проект рекомендуется разделить на 10 независимых задач. Если студентов меньше, часть задач можно объединить.

18. Задание студенту 1. Ядро системы и база данных

18.1. Цель работы

Разработать ядро системы, которое хранит заявки, статусы, файлы, результаты проверок и историю действий. Этот модуль является основой для интеграции всех остальных частей.

18.2. Функциональные требования

Студент должен реализовать:

- структуру проекта;
- модели данных;
- SQLite-базу;
- создание заявки;
- обновление статуса заявки;
- получение списка заявок;
- получение карточки заявки;
- сохранение путей к файлам;
- хранение результатов проверок;
- историю действий;
- единый сервис workflow;
- единый plugin registry для этапов обработки.

18.3. Основные сущности

- Conference;
- Issue;
- Submission;
- Author;
- SubmissionFile;
- CheckResult;
- WorkflowStageResult;
- StatusHistory;
- EditorDecision.

18.4. Минимальные функции

```
create_submission(data: dict) -> dict
get_submission(submission_id: str) -> dict
list_submissions(issue_id: str | None = None) -> list[dict]
update_submission_status(submission_id: str, status: str, comment: str) -> dict
save_check_result(submission_id: str, check_result: dict) -> None
add_event(submission_id: str, event_type: str, payload: dict) -> None
```

18.5. План работы на месяц

Неделя 1: создать структуру проекта, модели данных, SQLite-базу.

Неделя 2: реализовать сервис заявок и статусов.

Неделя 3: реализовать workflow engine и регистрацию этапов.

Неделя 4: интеграция с остальными модулями, тесты, README.

18.6. Критерии приёмки

Работа считается выполненной, если:

- база создаётся автоматически;
- можно создать заявку;
- можно изменить статус;
- можно получить список заявок;
- можно открыть карточку заявки;
- история действий сохраняется;
- результаты проверок сохраняются;
- есть тестовые данные;
- есть README;
- модуль используется другими студентами.

19. Задание студенту 1.доп. Интерфейс автора и загрузка материалов

19.1. Цель работы

Разработать пользовательский интерфейс, через который автор может подать материал на конференцию, загрузить DOCX-файл, посмотреть статус и получить результат обработки.

19.2. Функциональные требования

Студент должен реализовать:

- главную страницу конференции;
- страницу подачи материала;
- форму автора;
- форму метаданных материала;
- загрузку DOCX-файла;
- страницу статуса заявки;
- страницу просмотра замечаний;
- скачивание оформленного DOCX;
- скачивание PDF;
- кнопку подтверждения итогового варианта;
- загрузку исправленной версии.

19.3. Поля формы подачи

- ФИО контактного автора;
- e-mail;
- организация;
- название материала;
- секция конференции;
- ФИО соавторов;
- научный руководитель;
- аннотация, если автор хочет вставить отдельно;

ключевые слова;
файл DOCX.

19.4. Требования к интерфейсу

интерфейс должен быть простым;
ошибки должны отображаться понятным языком;
после загрузки автор должен видеть номер заявки;
автор должен видеть текущий статус;
автор не должен видеть внутренние технические логи;
при ошибке обработки автор должен получить понятное сообщение.

19.5. План работы на месяц

Неделя 1: прототип страниц подачи и статуса.

Неделя 2: подключение к ядру заявок.

Неделя 3: отображение отчёта проверок и файлов.

Неделя 4: сценарий согласования, тесты, README.

19.6. Критерии приёмки

автор может открыть страницу подачи;
автор может заполнить форму;
автор может загрузить DOCX;
после загрузки создаётся заявка;
автор видит номер заявки;
автор видит статус;
автор может скачать результат;
автор может подтвердить материал;
автор может загрузить исправленную версию.

20. Задание студенту 2. DOCX-извлечение и анализ структуры материала

20.1. Цель работы

Разработать модуль, который читает DOCX-файл, извлекает текст и структуру материала, определяет основные поля и сохраняет результат в JSON.

20.2. Функциональные требования

Модуль должен извлекать:

название;
авторов;
организацию;
аннотацию;
ключевые слова;
основной текст;

заголовки разделов;
таблицы;
подписи к таблицам;
рисунки;
подписи к рисункам;
список литературы.

20.3. Формат результата

```
{  
  "title": "Применение нейросетей в образовательных системах",  
  "authors": ["Иванов И.И.", "Петров П.П."],  
  "organization": "ТГТУ",  
  "abstract": "В работе рассматривается...",  
  "keywords": ["нейросети", "образование", "RAG"],  
  "body_text": "Основной текст...",  
  "sections": [  
    {  
      "title": "Введение",  
      "text": "..."  
    }  
  ],  
  "references": [  
    "..."  
  ],  
  "objects": {  
    "tables_count": 1,  
    "figures_count": 2,  
    "equations_count": 0  
  },  
  "warnings": []  
}
```

20.4. Методы извлечения

В MVP можно использовать несколько уровней:

по стилям Word;
по ключевым словам;
по регулярным выражениям;
по расположению текста;
с помощью LLM, если структура неоднозначна.

20.5. План работы на месяц

Неделя 1: чтение DOCX, извлечение всех абзацев и таблиц.

Неделя 2: определение названия, авторов, аннотации, ключевых слов.

Неделя 3: извлечение структуры разделов и списка литературы.

Неделя 4: интеграция, тесты на 10 разных DOCX, README.

20.6. Критерии приёмки

модуль принимает DOCX;
извлекает текст;
извлекает таблицы;
находит название;
находит авторов;
находит аннотацию;
находит ключевые слова;
формирует JSON;
при ошибке не падает;
показывает предупреждения;
есть тестовые DOCX.

21. Задание студенту 2 доп. Приведение материала к шаблону конференции

21.1. Цель работы

Разработать модуль, который берёт извлечённые данные и формирует новый DOCX по единому шаблону конференции.

21.2. Функциональные требования

Модуль должен:

принимать `extracted_metadata.json`;
открывать DOCX-шаблон конференции;
подставлять название;
подставлять авторов;
подставлять организацию;
подставлять научного руководителя;
подставлять аннотацию;
подставлять ключевые слова;
подставлять основной текст;
подставлять список литературы;
переносить таблицы, если возможно;
переносить рисунки, если возможно;
формировать `formatted_material.docx`;
формировать `formatting_report.json`.

21.3. Требования к шаблону

Шаблон должен содержать машинные поля:

```
{{ title_ru }}  
{{ authors }}  
{{ organization }}  
{{ supervisor }}
```

```
{{ abstract_ru }}
{{ keywords_ru }}
{{ body_text }}
{{ references }}
```

21.4. Формат отчёта оформления

```
{
  "status": "success",
  "template_used": "conference_template_v1.docx",
  "filled_fields": [
    "title_ru",
    "authors",
    "organization",
    "body_text"
  ],
  "missing_fields": [
    "abstract_ru"
  ],
  "warnings": [
    "Аннотация не найдена в исходном файле."
  ],
  "output_docx": "formatted_material.docx"
}
```

21.5. План работы на месяц

Неделя 1: создать DOCX-шаблон и тестовую подстановку.

Неделя 2: реализовать генерацию DOCX из JSON.

Неделя 3: добавить перенос таблиц, рисунков и списка литературы.

Неделя 4: интеграция с DOCX-извлечением, тесты, README.

21.6. Критерии приёмки

есть шаблон конференции;
модуль принимает JSON;
модуль создаёт DOCX;
DOCX открывается в Word/LibreOffice;
заполнены основные поля;
формируется отчёт;
ошибки отображаются понятно;
есть тестовый пример.

22. Задание студенту 3. PDF-экспорт и формирование файлов результата

22.1. Цель работы

Разработать модуль, который формирует PDF-версии материалов и готовит итоговый пакет файлов для автора и редактора.

22.2. Функциональные требования

Модуль должен:

- принимать `formatted_material.docx`;
- конвертировать DOCX в PDF;
- проверять, что PDF создан;
- сохранять PDF в папку заявки;
- формировать список доступных файлов;
- формировать ZIP-пакет результата;
- возвращать понятную ошибку, если PDF не создан.

22.3. Файлы результата

- `original.docx`;
- `formatted_material.docx`;
- `formatted_material.pdf`;
- `extracted_metadata.json`;
- `formatting_report.json`;
- `check_report.json`;
- `author_report.pdf` или `author_report.docx`.

22.4. План работы на месяц

Неделя 1: протестировать DOCX → PDF через LibreOffice.

Неделя 2: реализовать сервис экспорта PDF.

Неделя 3: добавить ZIP-пакет и проверку файлов.

Неделя 4: интеграция с авторским и редакторским интерфейсом.

22.5. Критерии приёмки

- DOCX конвертируется в PDF;
- PDF сохраняется в нужную папку;
- при ошибке есть понятное сообщение;
- можно скачать PDF;
- можно скачать ZIP;
- есть тесты;
- есть README.

23. Задание студенту 4. Модуль автоматических проверок на основе LLM

23.1. Цель работы

Разработать модуль, который запускает нейросетевые проверки материала и возвращает структурированный отчёт.

23.2. Проверки MVP

Нужно реализовать минимум 4 проверки:

проверка тематического соответствия;
смысловая проверка;
проверка научной структуры;
проверка качества формулировок.

23.3. Требования к результату

Каждая проверка должна возвращать JSON:

```
{
  "check_id": "semantic_quality_check",
  "status": "passed",
  "risk_level": "low",
  "score": 0.86,
  "summary": "Материал выглядит связным.",
  "warnings": [],
  "errors": [],
  "author_comment": "Существенных замечаний нет.",
  "editor_comment": "Можно передать на ручную проверку."
}
```

23.4. Требования к LLM-вызову

промты должны храниться отдельно;
ответ модели должен сохраняться;
ответ должен валидироваться как JSON;
при невалидном JSON должна быть повторная попытка;
при ошибке модели система не должна падать;
результат должен быть понятен редактору;
в отчёте нельзя скрывать предупреждения.

23.5. План работы на месяц

Неделя 1: подготовить промты и JSON-схемы.

Неделя 2: реализовать вызов модели и обработку ответа.

Неделя 3: реализовать 4 проверки.

Неделя 4: интеграция, тесты на нормальных и плохих текстах.

23.6. Критерии приёмки

модуль принимает текст материала;
запускает минимум 4 проверки;
возвращает JSON;
сохраняет raw-ответ модели;
обрабатывает ошибки;
показывает предупреждения;
интегрируется с карточкой заявки.

24. Задание студенту 4доп. Модуль проверок на недопустимое содержание и чувствительные сведения

24.1. Цель работы

Разработать отдельный модуль предварительной проверки текста на фрагменты, которые требуют обязательного внимания редактора перед публикацией.

24.2. Функциональные требования

Модуль должен проверять:

потенциально недопустимые призывы;
опасные инструкции;
оскорбительные формулировки;
дискриминационные формулировки;
личные данные;
контактные данные третьих лиц;
служебные сведения;
чрезмерно подробные технические сведения чувствительного характера;
непроверенные утверждения о людях и организациях.

24.3. Принцип работы

Модуль не должен делать окончательных выводов. Он должен только выделять фрагменты риска и отправлять их редактору.

24.4. Формат результата

```
{  
  "check_id": "restricted_information_check",  
  "status": "warning",  
  "risk_level": "medium",  
  "summary": "Найдены фрагменты, требующие ручной проверки.",  
  "flagged_fragments": [  
    {  
      "fragment": "пример фрагмента",  
      "risk_type": "personal_data",  
      "reason": "Возможное указание персональных данных.",  
    }  
  ]  
}
```

```
    "recommendation": "Проверить необходимость публикации."
  },
  ],
  "editor_comment": "Проверить отмеченные фрагменты перед принятием решения.",
  "author_comment": "В материале есть фрагменты, которые могут потребовать
уточнения."
}
```

24.5. План работы на месяц

Неделя 1: определить категории риска и JSON-схему.

Неделя 2: реализовать регулярные проверки для e-mail, телефонов, номеров документов.

Неделя 3: добавить LLM-проверку фрагментов.

Неделя 4: интеграция с общим отчётом, тесты, README.

24.6. Критерии приёмки

модуль принимает текст;
находит очевидные персональные данные;
находит подозрительные фрагменты;
возвращает JSON;
не блокирует материал автоматически;
передаёт предупреждения редактору;
есть тестовые примеры.

25. Задание студенту 3 доп. Интерфейс редактора

25.1. Цель работы

Разработать редакторскую панель, через которую человек может просматривать материалы, результаты проверок и принимать финальное решение.

25.2. Функциональные требования

Редакторская панель должна показывать:

список всех заявок;
фильтр по выпуску;
фильтр по статусу;
поиск по автору;
поиск по названию;
карточку материала;
метаданные материала;
исходный DOCX;
оформленный DOCX;
PDF;
результаты автоматических проверок;
историю действий;
кнопки решений.

25.3. Решения редактора

Редактор должен иметь кнопки:

- принять;
- отклонить;
- отправить на доработку;
- вернуть автору на согласование;
- включить в выпуск;
- исключить из выпуска.

25.4. Требования к карточке материала

Карточка должна содержать:

- номер заявки;
- статус;
- название;
- авторов;
- секцию;
- дату подачи;
- дату последнего изменения;
- общий риск автоматических проверок;
- краткую сводку замечаний;
- ссылки на файлы;
- полную историю обработки.

25.5. План работы на месяц

Неделя 1: список заявок и фильтры.

Неделя 2: карточка материала.

Неделя 3: кнопки решений и изменение статусов.

Неделя 4: интеграция с выпуском и проверками, тесты, README.

25.6. Критерии приёмки

- редактор видит список заявок;
- может открыть карточку;
- видит отчёты проверок;
- может скачать файлы;
- может принять материал;
- может отклонить материал;
- может отправить на доработку;
- решение сохраняется в истории.

26. Задание студенту 3 доп доп. Управление выпусками и сборка электронного сборника

26.1. Цель работы

Разработать модуль управления квартальными выпусками конференции и автоматической сборки электронного сборника из принятых материалов.

26.2. Функциональные требования

Модуль должен:

- создавать выпуск;
- задавать название выпуска;
- задавать год и квартал;
- показывать список принятых материалов;
- добавлять материал в выпуск;
- удалять материал из выпуска;
- сортировать материалы по секциям;
- формировать оглавление;
- формировать PDF-сборник;
- формировать страницу архива выпуска.

26.3. Структура выпуска

```
{
  "issue_id": "2026_q1",
  "title": "Искусственный интеллект и цифровые технологии: материалы
конференции",
  "year": 2026,
  "quarter": 1,
  "published_at": "2026-03-31",
  "sections": [
    {
      "title": "Искусственный интеллект в образовании",
      "submissions": [
        "SUB-2026-Q1-00001",
        "SUB-2026-Q1-00002"
      ]
    }
  ],
  "files": {
    "collection_pdf": "storage/issues/2026_q1/collection.pdf",
    "archive_page": "storage/issues/2026_q1/index.html"
  }
}
```

26.4. Требования к сборнику

Сборник должен содержать:

титульную страницу;
информацию о конференции;
состав организационного комитета, если задан;
оглавление;
разделы по секциям;
материалы авторов;
нумерацию страниц;
выходные данные;
дату выпуска.

26.5. План работы на месяц

Неделя 1: модель выпуска и интерфейс создания.

Неделя 2: добавление материалов в выпуск.

Неделя 3: генерация оглавления и сборника.

Неделя 4: публичный архив, тесты, README.

26.6. Критерии приёмки

можно создать выпуск;
можно добавить материалы;
можно отсортировать по секциям;
формируется PDF-сборник;
формируется страница выпуска;
файлы сохраняются в архиве;
есть тестовый выпуск из 5 материалов.

27. Задание студенту резерв. Уведомления, журнал событий, тестирование и демонстрационный стенд

27.1. Цель работы

Разработать вспомогательные модули уведомлений, логирования, тестовых данных и демонстрационного запуска всей системы.

27.2. Функциональные требования

Студент должен реализовать:

шаблоны уведомлений;
отправку уведомлений в тестовом режиме;
журнал событий;
журнал ошибок;
демонстрационные материалы;
скрипт инициализации тестовой базы;
единый README запуска;
интеграционный тест полного сценария.

27.3. Уведомления MVP

В MVP уведомления могут не отправляться реально по e-mail. Достаточно сохранять их в системе и показывать в интерфейсе.

Типы уведомлений:

- материал получен;
- материал оформлен;
- есть замечания;
- нужно согласование автора;
- материал отправлен редактору;
- материал принят;
- материал отправлен на доработку;
- материал включён в выпуск;
- выпуск опубликован.

27.4. План работы на месяц

Неделя 1: журнал событий и ошибок.

Неделя 2: шаблоны уведомлений.

Неделя 3: демонстрационные данные.

Неделя 4: интеграционный тест и инструкция запуска.

27.5. Критерии приёмки

- события сохраняются;
- ошибки сохраняются;
- уведомления формируются;
- есть тестовые заявки;
- есть тестовый выпуск;
- есть инструкция запуска;
- можно показать полный сценарий на демонстрации.

28. Общий интеграционный план на 1 месяц

Неделя 1. Проектирование и каркас

- согласовать структуру проекта;
- создать репозиторий;
- создать базовые модели;
- создать шаблон конференции;
- создать тестовые DOCX;
- подготовить каркас интерфейса;
- описать JSON-контракты;
- распределить зоны ответственности.

Результат недели:

проект запускается;
есть пустой интерфейс;
есть база;
есть модели;
есть тестовые файлы;
каждый студент понимает свой модуль.

Неделя 2. Реализация основных модулей

загрузка заявки;
сохранение DOCX;
извлечение структуры;
генерация DOCX по шаблону;
первые проверки;
редакторская карточка;
создание выпуска;
логирование событий.

Результат недели:

можно загрузить материал;
можно извлечь текст;
можно создать оформленный DOCX;
можно увидеть заявку в редакторской панели.

Неделя 3. Автоматизация и согласование

добавить LLM-проверки;
добавить отчёты замечаний;
добавить согласование автором;
добавить решения редактора;
добавить PDF-экспорт;
добавить включение материала в выпуск.

Результат недели:

материал проходит полный путь до решения редактора;
есть отчёт проверок;
есть PDF;
есть история статусов.

Неделя 4. Сборка выпуска и демонстрация

собрать PDF-сборник;
сделать публичный архив;
добавить тестовые данные;
исправить ошибки интеграции;
написать README;
подготовить демонстрационный сценарий;
провести финальное тестирование.

Результат недели:

есть рабочий MVP;
есть тестовый выпуск;

есть 5–10 тестовых материалов;
есть инструкция запуска;
есть демонстрация полного процесса.

29. Минимальный демонстрационный сценарий

Для защиты практики студенты должны показать:

1. Запуск системы одной командой.
 2. Открытие главной страницы конференции.
 3. Создание нового выпуска.
 4. Подача материала автором.
 5. Загрузка DOCX-файла.
 6. Извлечение структуры материала.
 7. Приведение материала к шаблону.
 8. Формирование PDF.
 9. Запуск автоматических проверок.
 10. Просмотр отчёта замечаний.
 11. Подтверждение материала автором.
 12. Просмотр материала редактором.
 13. Принятие материала редактором.
 14. Включение материала в выпуск.
 15. Сборка PDF-сборника.
 16. Публикация выпуска в архиве.
-

30. Тестовый набор данных

Для демонстрации нужно подготовить минимум 8 тестовых материалов:

1. Хорошо оформленный материал.
2. Материал без ключевых слов.
3. Материал без аннотации.
4. Материал с плохой структурой.
5. Материал с таблицей.
6. Материал с рисунком.
7. Материал со слабой смысловой связностью.
8. Материал с фрагментами, требующими ручной проверки.

Для каждого тестового материала нужно сохранить:

исходный DOCX;
ожидаемый extracted_metadata.json;
ожидаемый formatted_material.docx;
ожидаемый check_report.json;
краткое описание, что именно проверяется.

31. Общие критерии готовности MVP

Проект считается выполненным, если:

- система запускается локально;
- есть инструкция запуска;
- есть база данных;
- можно создать выпуск конференции;
- автор может подать материал;
- DOCX-файл сохраняется;
- структура материала извлекается;
- материал приводится к шаблону;
- формируется PDF;
- запускаются автоматические проверки;
- формируется отчёт замечаний;
- автор может подтвердить материал;
- редактор может принять или отклонить материал;
- принятый материал включается в выпуск;
- формируется электронный сборник;
- есть публичный архив выпуска;
- есть журнал событий;
- есть тестовые данные;
- каждый модуль имеет README;
- каждый студент может показать свой вклад.

32. Что должен сдать каждый студент

Каждый студент сдаёт:

- свой модуль в общей структуре проекта;
- README по своему модулю;
- описание функций;
- описание входных и выходных данных;
- тестовые примеры;
- скриншоты работы;
- список известных ограничений;
- краткий отчёт о выполненной работе;
- демонстрацию своего модуля.

33. Рекомендуемый формат отчёта студента

Отчёт должен содержать:

1. Название индивидуального задания.
2. Цель работы.
3. Описание реализованного модуля.

4. Используемые технологии.
 5. Структура файлов.
 6. Описание основных функций.
 7. Входные данные.
 8. Выходные данные.
 9. Примеры работы.
 10. Скриншоты интерфейса.
 11. Тестирование.
 12. Возникшие проблемы.
 13. Что не успел реализовать.
 14. Возможности дальнейшего развития.
 15. Вывод.
-

34. Главный принцип реализации

Система должна быть сделана не как набор разрозненных скриптов, а как единый конвейер обработки материалов конференции.

Каждый модуль должен быть подключаемым. Если один этап временно не работает, остальные части системы не должны полностью ломаться. Ошибки должны сохраняться в отчёте и отображаться пользователю понятным языком.

Главная ценность проекта — не отдельная DOCX-обработка и не отдельная нейросетевая проверка, а общий процесс:

принял материал
↓
привёл к шаблону
↓
проверил
↓
согласовал
↓
передал редактору
↓
включил в выпуск
↓
собрал сборник
↓
опубликовал в архиве

Именно этот процесс должен быть показан на финальной демонстрации.